

COPILOTTRAINING
TECHNICAL BRIEFING

GitHub Copilot + Azure Skills Plugin

Conversational Azure Prototyping and Deployment

From 45 minutes of portal clicks to 8 minutes of live-context conversation

Tech Talk · 45 min · Azure Developers · DevOps Engineers · Cloud Architects

🤔 THE CORE QUESTION

"How do I prototype and deploy Azure solutions conversationally?"

Azure developers spend 45+ minutes per cycle switching between portal, docs, and CLI

What if the AI already knew your subscription?

Azure Developer

Tab-switching between terminal, portal, and docs to understand current Azure state

DevOps Engineer

Scaffolding staging configs from production resources without copy-paste guesswork

Cloud Architect

Evaluating whether AI infrastructure tooling is a durable pattern — not a demo trick

45 min → 8 min

Prototype-to-deploy cycle with live Azure context vs. portal tab-switching

200+ live Azure tools

Direct SDK calls across 40+ services — resource queries, config reads, deployments

21 workflow skills

Curated deployment logic — azure-prepare, azure-validate, azure-deploy, and more

📖 TABLE OF CONTENTS

PART 1

Live Azure Context

Terminal as Azure interface

Query live subscription state from any conversation

PART 2

The Prototype Loop

Plan → Generate → Deploy

45-minute portal workflow compressed to 8 minutes

PART 3

Azure Skills Plugin

Three-layer stack explained

Why the loop works — Skills, MCP, and Foundry

PART 4

Guardrails

Trust boundaries for teams

Read-only by default, opt-in for writes

PART 1

Live Azure Context in the Terminal

Terminal replaces the portal — live subscription state in every conversation

Ask, Don't Click

Structured Live Output

Session Context Persists

```
copilot > List all storage accounts in my subscription
↳ 4 results — live, structured, in 3 seconds
```

From Tab-Switching to Terminal Conversation

Portal-First Discovery

Open Azure Portal, navigate to the resource group

Click through resources to find what you need

Copy values manually
SKUs, endpoints, networking rules to a text editor

Switch back to terminal — context lost between tools

Terminal-First Conversation

Ask: 'List storage accounts with SKUs and networking rules'

AI returns live structured output in seconds

Follow-up without re-querying
Context stays in session for subsequent questions

Scaffold new configs from live templates — same conversation

Live Resource Discovery in Action

Three queries, live Azure state, zero portal visits

```
$ copilot

> "List all storage accounts in my subscription with their locations and SKUs"

🤔 Copilot (Azure MCP):

  Found 4 storage accounts:
  proddata001 (East US) — Standard_GRS · public access disabled
  stagingdata001 (West US 2) — Standard_LRS · public blob enabled
  devlogs (East US 2) — Standard_LRS · public access disabled

> "Create a staging config like proddata001 but with LRS and staging-vnet"

🤔 Copilot:

  Based on proddata001 network config:
  SKU: Standard_LRS (downgraded — staging doesn't need geo-redundancy)
  VNet: staging-vnet/app-subnet (mapped from prod-vnet/app-subnet)
  TLS: 1.2 min · public access: disabled · firewall rules: retained

✅ Live template extracted — Bicep ready with one more prompt
```

0 portal visits · live config in under 60 seconds

Three Skills Power Every Deployment



azure-prepare

Project analysis, infra code gen, Dockerfile scaffolding — gets your project Azure-ready

- **Analyzes project structure**
- **Generates azd-compatible infra**
- **Creates Dockerfiles if needed**



azure-validate

Pre-deployment checks before touching anything — quotas, VNets, naming conflicts

- **Subscription quota checks**
- **VNet availability**
- **Naming conflict detection**



azure-deploy

Ordered deployment pipeline with rollback handling and context-aware error remediation

- **Correct execution order**
- **Rollback on failure**
- **Error context surfaced**

PART 2

The Prototype Loop

45-minute portal workflow → 8-minute conversational deployment with live context

⚖️ **Three Environments**

Plan Mode Validation

Bicep From Live Templates

```
copilot [plan] > Deploy a payments API like orders API with its own database
↳ 45 min → 8 min · zero portal visits
```

Same Intent. Three Environments.



No Skills (raw az)

Generic cloud advice, no Azure context, manual CLI commands — you fill all the gaps

- You write every az command
- No subscription awareness
- No workflow sequencing
- 45 min average cycle



Azure MCP Only

Live Azure data, correct tool calls — but no sequencing, no validation gates, no RBAC scope

- Queries live state ✓
- Ad hoc tool calls
- Wrong sequence risk
- ~15 min cycle



Skills + MCP

Correct calls in the correct sequence, validated pre-deployment, RBAC-aware from the start

- prepare → validate → deploy ✓
- Live state + workflow logic ✓
- RBAC validated upfront ✓
- 8 min cycle ✓

Traditional Workflow vs. Plan Mode

From trial-and-error to collaborative strategy before a line of code

❌ Traditional Workflow

1. **Describe what you want**
   Portal tab to check current state
2. **AI generates infrastructure**
   Assumes intent, one interpretation
3. **Deploy and discover mismatches**
   Wrong SKU, missing firewall rule, quota hit
4. **Fix and redeploy**
   Average 3-4 iterations before success

🔄 3-4 deploy attempts — 45 min average cycle

✅ With Plan Mode + Azure MCP

1. **Describe what you want**
   Shift+Tab to enter Plan Mode
2. **AI asks clarifying questions**
   Region? Shared plan? Database type? VNet?
3. **AI validates against live resources**
   ✅ VNet exists · ✅ quota available · ⚠️ firewall gap found
4. **Approve plan → generate Bicep**
   1-2 iterations · mismatches caught pre-deployment

✓ 1-2 attempts — 8 min with live validation

45 min → 8 min · misconfigurations caught in conversation, not post-deployment

Plan Mode in Action: Payments API Deployment

AI validates against live subscription state before a single line of infrastructure code

```
$ copilot [plan mode]
> "Deploy a new App Service for payments API — like orders API, own database"
🤔 Copilot (azure-prepare):
  A few questions to validate the approach:
  1. Region — orders API is East US. Co-locate or separate region?
  2. App Service Plan — share prod-asp-001 or dedicated for isolation?
  3. Database — new Azure SQL, Cosmos DB, or existing connection?
> "East US, dedicated plan, new Azure SQL with VNet firewall rules"
─────────────────────────────────────────────────────────────────
🔍 Copilot (azure-validate — live subscription check):
  ✅ East US S2 capacity available
  ✅ prod-vnet/payments-subnet exists (10.0.3.0/24, currently empty)
  ✅ Azure SQL quota: 8 of 20 databases used
  ⚠️ payments-subnet not in SQL firewall — will add automatically
✅ Plan approved → Bicep generated — payments-infrastructure.bicep
```

Subnet firewall gap caught pre-deployment · estimated cost ~$150/month surfaced

The Number That Travels to Management

Infrastructure-as-conversation — validated before deployment

8 min

prototype-to-deploy cycle with live Azure context

vs. 45-minute average with portal discovery + 3-4 failed deploys

✅ **1-2 iterations instead of 3-4**

Misconfigurations caught in conversation before touching Azure

📋 **Bicep from live templates**

Production-validated configs — not documentation examples with wrong SKUs

💰 **Cost estimate in the plan**

AI surfaces ~$150/month before you approve — no billing surprises

🔒 **RBAC gap caught pre-deploy**

azure-validate found the missing firewall rule before the deploy failed

💡 The loop works because the AI reads what's actually running, right now.

PART 3

Azure Skills Plugin: The Brain Layer

21 curated skills + 200+ live tools: the three-layer stack that made the loop work

🧠 **Skills as Workflow Brain**

🤲 **MCP as Execution Hands**

🤖 **Foundry as AI Specialist**

```
Plugin installed in .github/plugins/azure-skills/
↳ 21 skills · 200+ tools · 3 layers · 1 install
```

Why the Prototype Loop Worked

Three layers — Skills knew the sequence, MCP had the reach, Foundry handles AI

Brain	Azure Skills (21 curated) — workflow logic, decision trees, guardrails	`azure-prepare · azure-validate · azure-deploy`
Hands	Azure MCP Server (200+ tools, 40+ services) — live SDK calls to Azure	`Resource queries · Config reads · Deployments`
AI Specialist	Foundry MCP Server — AI model deployment and agent orchestration	`AI models · Endpoints · Agent workflows`

Skills encode the 'when' and 'why' — MCP provides the 'how' — Foundry handles the 'AI'

21 Skills Covering the Full Azure Lifecycle



Build + Deploy

Core deployment pipeline skills that activate in sequence for any Azure workload

- azure-prepare — project analysis + infra gen
- azure-validate — pre-deploy checks
- azure-deploy — ordered pipeline + rollback



Operate + Troubleshoot

Live telemetry and operational skills for diagnosing running workloads

- azure-diagnostics — AppInsights + logs
- azure-observability — alerting + dashboards
- azure-kusto — KQL query construction



Optimize + Govern

Cost, security, identity, and AI governance skills for production readiness

- azure-cost-optimization — right-sizing
- azure-rbac — RBAC + service principals
- azure-ai · azure-aigateway · microsoft-foundry

Install Once, Available Everywhere — ~10 Minutes

Install the Azure Skills Plugin

```
# Copilot CLI
/plugin marketplace add microsoft/azure-skills
/plugin install azure@azure-skills

# VS Code
# Install "Azure MCP" extension from Marketplace
# Skills Plugin is bundled and auto-configured

# Claude Code
/plugin marketplace add microsoft/azure-skills
/plugin install azure@azure-skills
```

🌐 Host-agnostic

Same skills work in Copilot CLI, VS Code, and Claude Code — install once, deploy everywhere

📁 Installs to .github/

Skills land in .github/plugins/azure-skills/ — version-controlled, team-shareable

🔌 Bundles all three layers

One install brings Azure Skills, Azure MCP Server, and Foundry MCP — no separate wiring

🔧 Extensible

Add org-specific skills alongside the 21 curated ones for internal conventions and governance

The Skills Layer Is Also Your First Line of Access Governance

MCP Only — No Governance Layer

AI executes tool calls with your full authenticated permissions

No workflow awareness — tool calls can happen in any order

No RBAC pre-validation

Permission errors surface post-execution, not pre-deployment

Trust depends entirely on the human reviewing outputs

Skills + MCP — azure-rbac Activated

azure-rbac skill validates role assignments before deployment steps

Principle of least privilege enforced as part of the workflow sequence

Service principal guidance built in

Skill recommends scoped SP over personal account for team workflows

Trust is architecturally enforced — not a human memory exercise

PART 4

Guardrails and Trust Boundaries

Read-only by default, opt-in for writes — enterprise-safe from day one



RBAC Inheritance



Read-Only First



Production Discipline

```
az ad sp create-for-rbac --role Reader --scopes /subscriptions/ ...
↳ Read-only by default · opt-in write access per service
```

The Safe On-Ramp: Start Read-Only

👁️ Reader Role — Start Here

Safe for production subscriptions
Query any resource, read any config — zero modification risk

Ideal for team onboarding
Entire team explores safely before governance decisions

Enable for: discovery, templating, config extraction
All the wow-moments work with Reader

Fails loudly on write attempts — accidental modification impossible

✏️ Contributor Role — Earn It

Use only in dedicated dev/staging subscriptions
Isolated from production — failures are contained

Require PR review before execution
AI-generated Bicep goes through standard approval

Establish conventions first
Custom instructions guide AI toward compliant resource patterns

Never use personal Owner account — create a scoped service principal

Explore → Prototype → Ship

Explore

Reader SP on any subscription — discover resources, extract configs, scaffold templates safely

- List resources + configs
- Extract live templates
- Generate Bicep for review
- No modification possible

Prototype

Contributor SP on isolated dev subscription — build, deploy, iterate with AI in a safe sandbox

- Dedicated dev subscription
- AI-generated configs reviewed
- Peer review before execution
- Production isolated

Ship

Approved patterns promoted to production — AI assists, humans approve, audit trail maintained

- Conventions established
- RBAC scoped per service
- PR-gated deployments
- Read-only by default

💡 CTO talking point: Read-only service principal by default — write access is opt-in per service.

THE VALUE RECAP

Before

- Open Azure Portal, navigate resource groups, find the resource
- Manually copy configuration values into a text editor or clipboard
- Search documentation for valid parameter ranges and SKU options
- Deploy, discover configuration mismatches, troubleshoot, redeploy

After

- Ask Copilot: 'What App Services are running and what are their configs?'
- AI returns live structured output with networking, settings, and SKUs
- Scaffold similar resource — AI uses live template, not generic docs example
- Plan Mode validates against subscription limits before deployment

45 min → 8 min
Prototype-to-deploy cycle

200+
Live Azure tools

21
Workflow skills

✅ WHAT YOU CAN DO TODAY

Today

- Install Azure MCP Server (npm, pip, or Docker — ~10 minutes)
- Run your first live query: 'List all storage accounts in my subscription'
- Try Plan Mode on a real deployment task with Shift+Tab

This Week

- Configure a read-only service principal for team-safe exploration
- Scaffold a staging environment config from a production resource
- Install the Azure Skills Plugin and run the prototype loop demo

This Month

- Evaluate org-wide rollout with RBAC-scoped service principals per team
- Build custom org-specific skills for your team's Azure workflow patterns
- Compare old portal workflow vs Skills+MCP on a real infrastructure task

🔑 Key Takeaway

Start with Reader role — the wow-moments all work before you grant write access.

REFERENCES

Official Documentation

Get started with the Azure MCP Server
Installation, authentication, and first queries

Best practices for GitHub Copilot CLI
Permissions, custom instructions, MCP integration

Azure Skills Plugin

Announcing the Azure Skills Plugin
21 curated skills, three-layer stack announcement

Azure Skills GitHub Repository
Source, examples, and community contributions

Tools and Source

Azure MCP Server GitHub Repository
200+ tools across 40+ Azure services



Infrastructure Becomes Conversational

GITHUB COPILOT + AZURE SKILLS PLUGIN

45 min → 8 min

Prototype-to-deploy cycle with live Azure context replacing portal tab-switching

21 Skills

Azure workflow logic encoded as Markdown — not generic prompts, governed deployment sequences

Read-Only First

Opt-in write access per service — enterprise-safe on-ramp before granting production access

How much of your Azure workflow still lives in the portal?